

Hands-On 15

Algoritmi Distribuiti: Flooding per Cammini Minimi

Sandi Russo

Corso di Laurea: Scienze Informatiche

21 maggio 2026

Indice

1	Introduzione	3
2	Definizione del Problema	3
3	Metodologia	4
3.1	Algoritmo di Flooding	4
3.2	Architettura Master-Slave	4
3.3	Comunicazione e Sincronizzazione	5
4	Presentazione dei Risultati	5
4.1	File di Configurazione e Utilità	5
4.2	Processo Principale (main.c)	6
4.3	Processo Master ('processo_master.c')	9
4.4	Processo Figlio ('processo_figlio.c')	10
4.5	Makefile	12
4.5.1	Output Atteso e Discussione	12
5	Conclusioni	13

1 Introduzione

Gli algoritmi distribuiti rappresentano una classe di algoritmi progettati per essere eseguiti su sistemi composti da più processori autonomi che comunicano tra loro scambiandosi messaggi. Questi sistemi sono onnipresenti, dalle reti di computer su larga scala ai sistemi multicore. Una delle sfide fondamentali in questo campo è il calcolo dei cammini minimi in un grafo, dove i nodi del grafo rappresentano i processori e gli archi i canali di comunicazione.

Questa relazione si concentra sull'implementazione di un algoritmo distribuito per trovare i cammini minimi da un singolo nodo sorgente a tutti gli altri nodi in un grafo. L'approccio utilizzato è una variante dell'algoritmo di Bellman-Ford, comunemente nota come **flooding**, dove le informazioni sulla distanza si propagano attraverso la rete in ondate successive. Per simulare un ambiente distribuito su una singola macchina, abbiamo adottato un modello basato su processi POSIX, dove ogni nodo del grafo è rappresentato da un processo figlio distinto. La comunicazione tra i nodi (processi) è realizzata tramite **named pipes (FIFO)**, che fungono da canali di comunicazione unidirezionali.

Un aspetto cruciale di molti algoritmi distribuiti è la **sincronizzazione**. Per emulare un sistema sincrono, in cui l'algoritmo procede in "round" discreti, è stato introdotto un processo **master**. Il suo unico scopo è orchestrare l'esecuzione dei processi figli, garantendo che tutti i nodi completino una fase del round (ad esempio, l'invio di messaggi) prima di passare alla fase successiva (l'elaborazione dei messaggi ricevuti). Questa sincronizzazione è implementata utilizzando un meccanismo di barriera basato su FIFO.

Verrà presentata la struttura modulare del codice, suddivisa in file per la configurazione ('config.h'), la logica del master ('processo_master.c'), la logica dei nodi figli ('processo_figlio.c'), il punto di ingresso principale ('main.c') e le funzioni di utilità ('utils.c'). L'obiettivo è fornire una spiegazione chiara del funzionamento di ogni componente, analizzando le funzioni chiave, il protocollo di comunicazione e sincronizzazione, e illustrando il comportamento del sistema attraverso l'analisi dell'output atteso, che culmina nella costruzione dell'albero dei cammini minimi.

2 Definizione del Problema

Il problema consiste nel calcolare l'albero dei cammini minimi da un nodo sorgente designato (la "radice", che nel nostro caso sarà il Nodo 1) a tutti gli altri nodi in un grafo orientato e non pesato. Un "cammino minimo" è definito come il percorso con il minor numero di archi (o "salti").

L'implementazione deve aderire ai seguenti vincoli e requisiti, tipici di un sistema distribuito:

- **Modello Distribuito:** L'algoritmo deve essere implementato come un insieme di processi concorrenti, dove ogni processo rappresenta un nodo del grafo.
- **Conoscenza Locale:** Ogni processo-nodo possiede solo informazioni locali: il proprio ID, la sua attuale stima della distanza minima dalla radice, il suo attuale "padre" nel cammino minimo, e l'elenco dei suoi vicini immediati (a cui può inviare messaggi e da cui può riceverne). La topologia globale del grafo non è nota a nessun singolo nodo.
- **Comunicazione via Messaggi:** I processi-nodo possono comunicare esclusivamente scambiandosi messaggi attraverso canali predefiniti, che corrispondono agli archi del grafo. Nel nostro caso, questi canali sono implementati come named pipes (FIFO).

- **Esecuzione Sincrona:** L'algoritmo deve procedere in round discreti e sincronizzati. Un processo master deve garantire che nessun nodo inizi il round $k + 1$ prima che tutti i nodi abbiano completato il round k . Ogni round è ulteriormente suddiviso in una fase di invio di messaggi e una fase di ricezione/elaborazione, anch'esse sincronizzate.

L'obiettivo finale è che, alla terminazione dell'algoritmo, ogni processo-nodo abbia determinato la sua distanza minima dalla radice e l'ID del nodo vicino che funge da suo genitore nel cammino minimo. Il programma principale dovrà quindi raccogliere queste informazioni e ricostruire e visualizzare l'intero albero dei cammini minimi.

3 Metodologia

La soluzione è stata sviluppata in linguaggio C su un sistema POSIX-compatibile, sfruttando processi e named pipes (FIFO) per simulare un sistema distribuito. La metodologia si articola su tre pilastri: l'algoritmo di flooding, l'architettura dei processi (master-slave) e il meccanismo di comunicazione e sincronizzazione.

3.1 Algoritmo di Flooding

L'algoritmo si basa su un principio iterativo simile a quello di Bellman-Ford. Ogni nodo 'i' mantiene due variabili di stato principali: 'distanza_minima_i' (la lunghezza del cammino più breve conosciuto dalla radice a 'i') e 'parent_i' (il vicino che si trova su tale cammino).

1. **Inizializzazione:** La radice (Nodo 1) imposta la sua distanza a 0 e il suo genitore a se stessa. Tutti gli altri nodi inizializzano la loro distanza a infinito e il genitore a un valore nullo.
2. **Round Sincroni:** L'algoritmo procede per un numero di round pari al diametro del grafo più uno (per garanzia di convergenza). Ogni round consiste in due fasi sincrone:
 - **Fase di Invio (msg):** Ogni nodo che ha migliorato la propria distanza nel round precedente invia un messaggio a tutti i suoi vicini uscenti. Il messaggio contiene la sua nuova distanza minima dalla radice. Questo "inonda" la rete con le informazioni aggiornate.
 - **Fase di Elaborazione (stf - State Transition Function):** Ogni nodo ascolta i messaggi in arrivo dai suoi vicini entranti. Per ogni messaggio ricevuto da un vicino 'j' con distanza 'd_j', il nodo calcola una distanza potenziale 'd_j + 1'. Se questa distanza è inferiore alla sua 'distanza_minima' attuale, aggiorna la sua distanza e imposta il suo 'parent' a 'j'.

Dopo 'DIAMETRO + 1' round, ogni nodo avrà scoperto il suo cammino minimo dalla radice.

3.2 Architettura Master-Slave

Il sistema è composto da un processo **master** e 'NUM_NODI' processi **figli** (o slave).

- **Processo Master ('processo_master.c'):** Il suo unico ruolo è la sincronizzazione. Non partecipa all'algoritmo di calcolo. Utilizza due FIFO principali, 'FIFO_MASTER' e 'FIFO_CODA', per implementare una barriera. Per avviare una fase, scrive un segnale (un

singolo carattere) su 'FIFO_MASTER' per ogni figlio. Successivamente, attende di leggere un messaggio di conferma da ogni figlio su 'FIFO_CODA'. Solo quando tutte le conferme sono arrivate, la fase è considerata conclusa e il master può avviare la successiva.

- **Processo Figlio ('processo_figlio.c'):** Ogni figlio esegue la logica di un nodo del grafo. Inizialmente, apre le FIFO per comunicare con i suoi vicini. Poi, entra nel ciclo principale dei round, dove attende i segnali di sincronizzazione dal master per eseguire le fasi 'msg' e 'stf'. Al termine della simulazione, scrive il suo risultato (la coppia 'id, parent') su un file condiviso ('Dati_figli.txt').

3.3 Comunicazione e Sincronizzazione

- **Canali del Grafo (FIFO):** La comunicazione tra nodi avviene tramite FIFO dedicate. Per ogni arco orientato da un nodo 'A' a un nodo 'B' nel grafo, viene creata una FIFO nominata (es. 'fifo_FROM_A_TO_B'). Il nodo 'A' aprirà questa FIFO in scrittura, mentre 'B' la aprirà in lettura. Per evitare deadlock durante l'apertura delle FIFO (poiché l'apertura in scrittura blocca finché non c'è un lettore e viceversa), il master sincronizza anche la fase di setup, garantendo che tutti i nodi aprano prima i loro canali di lettura e solo dopo i loro canali di scrittura.
- **Sincronizzazione dei Round:** La funzione 'orchestra_fase' nel master e la funzione 'attendi_e_segna' nel figlio implementano la barriera. Il master invia un segnale specifico per ogni fase (es. 'R' per Read-setup, 'W' per Write-setup, 'S' per Send, 'P' per Process), e i figli attendono quel segnale specifico prima di procedere e inviare una conferma al termine.

Infine, la compilazione del progetto modulare è gestita da un 'Makefile', che automatizza la creazione dei file oggetto e il linking finale per produrre un singolo eseguibile.

4 Presentazione dei Risultati

In questa sezione, analizziamo il codice sorgente di ciascun modulo e discutiamo il suo ruolo nell'architettura complessiva della simulazione.

4.1 File di Configurazione e Utilità

Il file 'config.h' centralizza tutte le costanti, le strutture dati e i prototipi di funzione, garantendo coerenza tra i vari moduli.

```
1 #ifndef CONFIG_H
2 #define CONFIG_H
3
4 #include <stdio.h>
5 #include <stdlib.h>
6 #include <unistd.h>
7 #include <sys/types.h>
8 #include <sys/stat.h>
9 #include <sys/wait.h>
10 #include <fcntl.h>
11 #include <string.h>
12 #include <errno.h>
```

```

13
14 #define NUM_NODI 6
15 #define DIAMETRO 3
16 #define INFINITO (NUM_NODI + 1)
17
18 #define FIFO_MASTER "fifo_master"
19 #define FIFO_CODA "fifo_coda"
20 #define FIFO_PATH_FORMAT "fifo_FROM_%d_TO_%d"
21
22 typedef struct {
23     int id_mittente_originario; // radice (sempre 1)
24     int distanza; // distanza tra mittente e radice
25 } Messaggio;
26
27 // Matrice di adiacenza del grafo
28 extern int grafo[NUM_NODI][NUM_NODI];
29
30 void processo_master();
31 void processo_figlio(int id_nodo);
32 void msg(int mio_id, int distanza_da_inviare, int fds_scrittura[NUM_NODI]);
33 void stf(int mio_id, int *parent, int *distanza_minima, int *stato_cambiato
    , int fds_lettura[NUM_NODI]);
34 void die(const char *s);
35
36 #endif // CONFIG_H

```

Listing 1: Codice sorgente config.h (Configurazione Globale)

Questo header definisce parametri cruciali come ‘NUM_NODI’ e ‘DIAMETRO’, i nomi delle FIFO di sincronizzazione e il formato per quelle di comunicazione. Definisce la struct ‘Messaggio’ usata per la comunicazione tra nodi e dichiara la matrice di adiacenza ‘grafo’ come ‘extern’ per renderla accessibile da tutti i file sorgente che includono l’header, pur essendo definita in un solo punto (‘main.c’). Infine, elenca i prototipi di tutte le funzioni principali.

```

1 #include "config.h"
2 // lo utilizzo per stampare errori e terminare il programma
3 void die(const char *s) {
4     perror(s);
5     exit(1);
6 }

```

Listing 2: Codice sorgente utils.c (Funzioni di Utilità)

Il file ‘utils.c’ contiene la semplice ma fondamentale funzione ‘die’, utilizzata in tutto il progetto per gestire errori critici in modo pulito, stampando un messaggio di errore e terminando l’esecuzione.

4.2 Processo Principale (main.c)

Il file ‘main.c’ è il punto d’ingresso del programma. È responsabile della definizione della topologia del grafo, della creazione e pulizia delle FIFO, della creazione dei processi figli e dell’avvio del processo master.

```

1 #include "config.h"
2
3 int grafo[NUM_NODI][NUM_NODI] = {
4     {0, 1, 1, 1, 0, 0}, // 1 -> 2, 3, 4

```

```

5     {0, 0, 1, 1, 1, 0}, // 2 -> 3, 4, 5
6     {0, 0, 0, 1, 1, 0}, // 3 -> 4, 5
7     {0, 0, 0, 0, 0, 0}, // 4 -> nessuno
8     {1, 0, 0, 0, 0, 1}, // 5 -> 1, 6
9     {0, 0, 0, 1, 1, 0} // 6 -> 4, 5
10 };
11
12 void crea_fifo() {
13     unlink(FIFO_MASTER); if (mkfifo(FIFO_MASTER, 0666) == -1) die("mkfifo
14     master");
15     unlink(FIFO_CODA); if (mkfifo(FIFO_CODA, 0666) == -1) die("mkfifo coda"
16 );
17     char nome_fifo[25];
18     for (int i = 0; i < NUM_NODI; ++i) {
19         for (int j = 0; j < NUM_NODI; ++j) {
20             if (grafo[i][j] == 1) {
21                 sprintf(nome_fifo, FIFO_PATH_FORMAT, i + 1, j + 1);
22                 unlink(nome_fifo);
23                 if (mkfifo(nome_fifo, 0666) == -1) die("mkfifo canale");
24             }
25         }
26     }
27 }
28
29 void pulisci_fifo() {
30     unlink(FIFO_MASTER); unlink(FIFO_CODA);
31     char nome_fifo[25];
32     for (int i = 0; i < NUM_NODI; ++i) {
33         for (int j = 0; j < NUM_NODI; ++j) {
34             if (grafo[i][j] == 1) { sprintf(nome_fifo, FIFO_PATH_FORMAT, i
35 + 1, j + 1); unlink(nome_fifo); }
36         }
37     }
38 }
39
40 void mostra_risultati() {
41     printf("\n--- Albero dei Cammini Minimi dalla Radice (Nodo 1) ---\n");
42     FILE *f = fopen("Dati_figli.txt", "r");
43     if (!f) { die("Impossibile leggere il file dei risultati"); }
44
45     int parents[NUM_NODI];
46     int id_figlio, id_padre;
47
48     // inizializzo l'array dei padri
49     for (int i = 0; i < NUM_NODI; ++i) { parents[i] = -1; }
50
51     // Leggo le coppie (figlio, padre) dal file e popola l'array
52     while (fscanf(f, "%d %d\n", &id_figlio, &id_padre) == 2) {
53         if (id_figlio > 0 && id_figlio <= NUM_NODI) {
54             parents[id_figlio - 1] = id_padre;
55         }
56     }
57     fclose(f);
58
59     // itera su ogni nodo per stamparne il percorso
60     for (int i = 0; i < NUM_NODI; ++i) { // i e' l'indice (0-based)
61         printf("Percorso per Nodo %d: ", i + 1);
62
63         if (parents[i] == -1) {

```

```

60         printf("Non raggiungibile\n");
61         continue;
62     }
63
64     int percorso[NUM_NODI];
65     int len = 0;
66     int curr_idx = i;
67
68     // Ricostruisci il percorso all'indietro
69     while (curr_idx != -1) {
70         percorso[len++] = curr_idx + 1; // salva il numero del nodo
71         if (curr_idx + 1 == parents[curr_idx]) break; // qui arriviamo
72     alla radice
73         curr_idx = parents[curr_idx] - 1; // va all'indice del padre
74     }
75
76     for (int j = len - 1; j >= 0; --j) { // lo uso per la stampa del
77     percorso
78         printf("%d%s", percorso[j], (j > 0) ? " -> " : "\n");
79     }
80     remove("Dati_figli.txt");
81 }
82
83 int main() {
84     pid_t pids[NUM_NODI];
85     remove("Dati_figli.txt");
86     printf("Creazione delle FIFO...\n");
87     crea_fifo();
88     printf("Creazione dei processi figli (Nodi)...\n");
89     for (int i = 0; i < NUM_NODI; ++i) {
90         pids[i] = fork();
91         if (pids[i] < 0) die("fork");
92         if (pids[i] == 0) { processo_figlio(i + 1); exit(0); }
93     }
94     printf("Avvio del processo Master...\n");
95     processo_master();
96     printf("Master ha terminato. In attesa dei figli...\n");
97     for (int i = 0; i < NUM_NODI; ++i) { wait(NULL); }
98     printf("Tutti i figli hanno terminato.\n");
99     mostra_risultati();
100     printf("Pulizia delle FIFO...\n");
101     pulisci_fifo();
102     return 0;
103 }

```

Listing 3: Codice sorgente main.c (Orchestrazione Generale)

Il ‘main’ definisce la matrice ‘grafo’, che rappresenta la topologia della rete. Il suo flusso è lineare: pulisce eventuali file di output residui, crea tutte le FIFO necessarie, esegue un ciclo ‘fork’ per generare i ‘NUM_NODI’ processi figli (ciascuno dei quali invocherà ‘processo_figlio’), dopodiché il processo genitore originale assume il ruolo di master invocando ‘processo_master’. Una volta che il master ha finito di sincronizzare i round, il genitore attende la terminazione di tutti i figli con ‘wait’. Infine, chiama ‘mostra_risultati’ per leggere il file di output ‘Dati_figli.txt’, ricostruire i percorsi e stamparli, per poi terminare pulendo le FIFO.

4.3 Processo Master ('processo_master.c')

Questo modulo contiene la logica del processo di sincronizzazione, che agisce come un direttore d'orchestra per i nodi.

```
1 #include "config.h"
2
3 void orchestra_fase(const char* nome_fase, char segnale) {
4     // invio il segnale di avvio della fase a tutti i nodi
5     int fifo_master_fd = open(FIFO_MASTER, O_WRONLY);
6     if (fifo_master_fd < 0) die("master: open fifo_master");
7     for (int i = 0; i < NUM_NODI; ++i) {
8         if (write(fifo_master_fd, &segnale, sizeof(char)) != sizeof(char))
9     {
10         perror("master: write segnale");
11     }
12     }
13     close(fifo_master_fd);
14
15     // attendo la conferma da parte di tutti i nodi
16     int fifo_coda_fd = open(FIFO_CODA, O_RDONLY);
17     if (fifo_coda_fd < 0) die("master: open fifo_coda");
18     int nodi_completati = 0;
19     int id_nodo_letto;
20     while (nodi_completati < NUM_NODI) {
21         if (read(fifo_coda_fd, &id_nodo_letto, sizeof(int)) == sizeof(int))
22     {
23         nodi_completati++;
24     }
25     }
26     close(fifo_coda_fd);
27     printf("MASTER: Fase '%s' completata da tutti i nodi.\n", nome_fase);
28 }
29
30 void processo_master() {
31     orchestra_fase("Apertura Canali di Lettura", 'R');
32     orchestra_fase("Apertura Canali di Scrittura", 'W');
33
34     // Loop per i round
35     int round_totali = DIAMETRO + 1;
36     for (int round = 1; round <= round_totali; ++round) {
37         printf("\n--- MASTER: Inizio Round %d ---\n", round);
38         orchestra_fase("Invio Messaggi", 'S'); // 'S' per Send
39         orchestra_fase("Elaborazione Messaggi", 'P'); // 'P' per Process
40     }
41 }
```

Listing 4: Codice sorgente processo_master.c (Logica di Sincronizzazione)

La funzione chiave qui è 'orchestra_fase'. Essa implementa una barriera di sincronizzazione: prima invia un segnale a tutti i figli attraverso 'FIFO_MASTER', poi si mette in attesa su 'FIFO_CODA' finché non ha ricevuto una conferma da ciascuno dei 'NUM_NODI' figli. La funzione 'processo_master' utilizza 'orchestra_fase' per scandire l'intera simulazione: prima le due fasi di setup per l'apertura delle FIFO, poi un ciclo che per ogni round orchestra la fase di invio ('S') e quella di elaborazione ('P').

4.4 Processo Figlio ('processo_figlio.c')

Questo è il cuore dell'algoritmo distribuito, dove viene eseguita la logica di ciascun nodo.

```
1 #include "config.h"
2
3 void msg(int mio_id, int distanza_da_inviare, int fds_scrivtura[NUM_NODI])
4 {
5     Messaggio m = {1, distanza_da_inviare};
6     printf("NODO %d [msg]: Propago la mia distanza migliore (%d).\n",
7     mio_id, m.distanza);
8     for (int i = 0; i < NUM_NODI; ++i) {
9         if (grafo[mio_id - 1][i] == 1 && fds_scrivtura[i] != -1) {
10             write(fds_scrivtura[i], &m, sizeof(Messaggio));
11         }
12     }
13 }
14
15 void stf(int mio_id, int *parent, int *distanza_minima, int *stato_cambiato,
16 int fds_lettura[NUM_NODI]) {
17     Messaggio m;
18     *stato_cambiato = 0;
19     for (int i = 0; i < NUM_NODI; ++i) {
20         if (grafo[i][mio_id - 1] == 1 && fds_lettura[i] != -1) {
21             int mittente_immediato = i + 1;
22             while (read(fds_lettura[i], &m, sizeof(Messaggio)) > 0) {
23                 int distanza_proposta = m.distanza + 1;
24                 if (distanza_proposta < *distanza_minima) {
25                     printf("NODO %d [stf]: CAMMINO MIGLIORE trovato via
26 NODO %d! Distanza %d -> %d\n", mio_id, mittente_immediato, *
27 distanza_minima, distanza_proposta);
28                     *distanza_minima = distanza_proposta;
29                     *parent = mittente_immediato;
30                     *stato_cambiato = 1;
31                 }
32             }
33         }
34     }
35 }
36
37 void attendi_e_segna(int id_nodo, char segnale_atteso) {
38     char received_signal;
39     int fifo_master_fd = open(FIFO_MASTER, O_RDONLY);
40     if(fifo_master_fd < 0) die("figlio: open fifo_master");
41
42     if (read(fifo_master_fd, &received_signal, sizeof(char)) < 0) {
43         die("figlio: read from fifo_master");
44     }
45
46     if(received_signal != segnale_atteso){
47         fprintf(stderr, "ERRORE DI SINCRONIZZAZIONE: Nodo %d si aspettava
48 '%c' ma ha ricevuto '%c'\n", id_nodo, segnale_atteso, received_signal);
49         exit(EXIT_FAILURE);
50     }
51
52     close(fifo_master_fd);
53
54     int fifo_coda_fd = open(FIFO_CODA, O_WRONLY);
55     if(fifo_coda_fd < 0) die("figlio: open fifo_coda");
```

```

50     write(fifo_coda_fd, &id_nodo, sizeof(int));
51     close(fifo_coda_fd);
52 }
53
54 void processo_figlio(int id_nodo) {
55     int parent = -1;
56     int distanza_minima = INFINITO;
57     int stato_cambiato = 0;
58
59     if (id_nodo == 1) { parent = 1; distanza_minima = 0; stato_cambiato =
60     1; }
61
62     int fds_lettura[NUM_NODI], fds_scrittura[NUM_NODI];
63     for (int i = 0; i < NUM_NODI; ++i) { fds_lettura[i] = -1; fds_scrittura
64     [i] = -1; }
65
66     attendi_e_segnaia(id_nodo, 'R');
67     for (int i = 0; i < NUM_NODI; ++i) { if (grafo[i][id_nodo - 1] == 1) {
68     char n[30]; sprintf(n, FIFO_PATH_FORMAT, i + 1, id_nodo); fds_lettura[i]
69     = open(n, O_RDONLY | O_NONBLOCK); } }
70     attendi_e_segnaia(id_nodo, 'W');
71     for (int j = 0; j < NUM_NODI; ++j) { if (grafo[id_nodo - 1][j] == 1) {
72     char n[30]; sprintf(n, FIFO_PATH_FORMAT, id_nodo, j + 1); fds_scrittura[
73     j] = open(n, O_WRONLY); } }
74
75     for (int round = 1; round <= DIAMETRO + 1; ++round) {
76         attendi_e_segnaia(id_nodo, 'S');
77         if (stato_cambiato) {
78             msg(id_nodo, distanza_minima, fds_scrittura);
79         }
80         attendi_e_segnaia(id_nodo, 'P');
81         stf(id_nodo, &parent, &distanza_minima, &stato_cambiato,
82         fds_lettura);
83     }
84
85     for (int i = 0; i < NUM_NODI; ++i) { if (fds_lettura[i] != -1) close(
86     fds_lettura[i]); if (fds_scrittura[i] != -1) close(fds_scrittura[i]); }
87     FILE *f = fopen("Dati_figli.txt", "a");
88     if (f) {
89         fprintf(f, "%d %d\n", id_nodo, parent);
90         fclose(f);
91     }
92 }

```

Listing 5: Codice sorgente processo_figlio.c (Logica del Nodo)

Il ‘processo_figlio’ inizializza il suo stato (distanza e genitore; la radice ha valori speciali). La funzione ‘attendi_e_segnaia’ è il complemento di ‘orchestra_fase’: attende un segnale dal master su ‘FIFO_MASTER’ e invia una conferma su ‘FIFO_CODA’. Il flusso principale del figlio segue esattamente le fasi dettate dal master: attende il segnale ‘R’, apre i canali di lettura; attende ‘W’, apre i canali di scrittura. Poi entra nel ciclo dei round: attende ‘S’, esegue ‘msg’ (se il suo stato è cambiato); attende ‘P’, esegue ‘stf’ per aggiornare il suo stato in base ai messaggi ricevuti. Al termine, chiude tutti i file descriptor e appende il suo risultato (la coppia ‘id, parent’) al file ‘Dati_figli.txt’.

4.5 Makefile

Infine, il ‘Makefile’ gestisce la compilazione di tutti i moduli in un unico eseguibile.

```
1 # Makefile per il progetto Flooding
2
3 # Definisce il compilatore C da utilizzare
4 CC = gcc
5 # Definisce i flag da passare al compilatore
6 CFLAGS = -g -Wall
7 # Definisce il nome dell'eseguibile finale
8 TARGET = flooding_sim
9
10 # Elenco dei file oggetto (.o) necessari
11 OBJS = main.o processo_master.o processo_figlio.o utils.o
12
13 # Regola di default: 'all' dipende dall'eseguibile
14 all: $(TARGET)
15
16 # Regola per creare l'eseguibile: collega tutti i file oggetto
17 $(TARGET): $(OBJS)
18     $(CC) $(CFLAGS) -o $@ $^
19
20 # Regola Pattern per compilare un file sorgente .c in un file oggetto .o
21 # Ogni .o dipende dal suo .c e dall'header comune 'config.h'
22 %.o: %.c config.h
23     $(CC) $(CFLAGS) -c $< -o $@
24
25 # Regola 'clean': rimuove i file generati
26 clean:
27     rm -f $(OBJS) $(TARGET) Dati_figli.txt
28     @echo "File compilati e di output rimossi."
29
30 # Dichiarazione di target 'PHONY'
31 .PHONY: all clean
```

Listing 6: Makefile per la Compilazione del Progetto

Questo ‘Makefile’ definisce le variabili per il compilatore (‘CC’), i flag (‘CFLAGS’), il nome dell’eseguibile (‘TARGET’) e l’elenco dei file oggetto (‘OBJS’). La regola ‘all’ costruisce il target. La regola di linking (‘*TARGET*’):(OBJS)’ unisce tutti i file ‘.o’. La regola pattern ‘

4.5.1 Output Atteso e Discussione

Eseguendo ‘./flooding_sim’ dopo la compilazione con ‘make’, l’output mostrerà la sequenza di operazioni. Inizialmente, si vedranno i messaggi di setup da ‘main’. Poi, l’output del master scandirà l’esecuzione, annunciando l’inizio e la fine di ogni fase e di ogni round. Tra i messaggi del master, appariranno in modo non deterministico i log dei processi figli, in particolare i messaggi ‘[stf]: CAMMINO MIGLIORE...’ che indicano la convergenza dell’algoritmo.

L’output finale e più importante è quello generato dalla funzione ‘mostra_risultati’, che avrà un aspetto simile a questo:

```
Percorso per Nodo 1: 1
Percorso per Nodo 2: 1 -> 2
Percorso per Nodo 3: 1 -> 3
Percorso per Nodo 4: 1 -> 4
```

Percorso per Nodo 5: 1 -> 2 -> 5

Percorso per Nodo 6: 1 -> 2 -> 5 -> 6

Questo risultato dimostra che l'algoritmo ha calcolato correttamente i cammini più brevi dalla radice (Nodo 1) a tutti gli altri nodi, ricostruendo l'albero dei cammini minimi basandosi sulle relazioni padre-figlio scoperte da ciascun nodo in modo distribuito. L'output conferma il successo della simulazione e la correttezza dell'algoritmo di flooding implementato.

5 Conclusioni

Questo progetto ha permesso di implementare e analizzare con successo una simulazione di un algoritmo distribuito per il calcolo dei cammini minimi, basato sul principio del flooding. Attraverso l'uso di processi POSIX e named pipes (FIFO), è stato possibile modellare un sistema di nodi concorrenti che comunicano scambiandosi messaggi, riflettendo le condizioni di un vero sistema distribuito.

L'architettura **master-slave** si è rivelata una soluzione efficace per emulare un'esecuzione **sincrona**, un requisito fondamentale per molti algoritmi distribuiti basati su round. Il master, agendo come un semplice orchestratore, ha permesso di separare nettamente la logica di sincronizzazione da quella algoritmica, eseguita interamente dai processi figli. Questo ha migliorato la modularità e la chiarezza del codice.

La gestione della comunicazione tramite **FIFO** ha messo in luce alcune delle sfide pratiche della programmazione inter-processo, come la necessità di evitare deadlock durante l'inizializzazione dei canali di comunicazione, risolta attraverso una fase di setup sincronizzata. L'uso di un file header comune ('config.h') e di un 'Makefile' ha rafforzato le buone pratiche di sviluppo, garantendo manutenibilità e facilità di compilazione.

In conclusione, l'esercizio ha fornito una comprensione pratica e approfondita dei concetti chiave della programmazione distribuita: la gestione dello stato locale, la comunicazione basata su messaggi, e l'importanza critica della sincronizzazione. La soluzione sviluppata non solo risolve correttamente il problema posto, ma costituisce anche un modello robusto e scalabile per la simulazione di altri algoritmi distribuiti sincroni.